

《微机原理实验》实验报告

实验名称：

I/O 接口实验

指导教师：

肖山林

姓名：

徐睿琳

学号：

23342107

专业/班级：

微电子/三班

分组序号：

A412-A05

实验日期：

2025.12.08

实验地点：

教学楼 A412

是否调课/补课：

否

成绩：

目录

1	实验内容与设计	2
1.1	I/O 端口控制与流水灯	2
1.1.1	电路原理图与代码流程	2
1.2	LCD 字符显示器控制	4
1.2.1	电路原理图与代码逐行讲解	4
2	实验结果	6
2.1	I/O 端口控制与流水灯	6
2.2	LCD 字符显示器控制	7
3	思考与总结	8
3.1	扩展与鲁棒性提升	8
3.1.1	硬件扩展方案	8
3.1.2	软件鲁棒性改进	8
3.1.3	故障诊断与保护机制	8
3.2	思考与展望	8
	附录 A 增强型 I/O 控制程序	9

1 实验内容与设计

1.1 I/O 端口控制与流水灯

1.1.1 电路原理图与代码流程

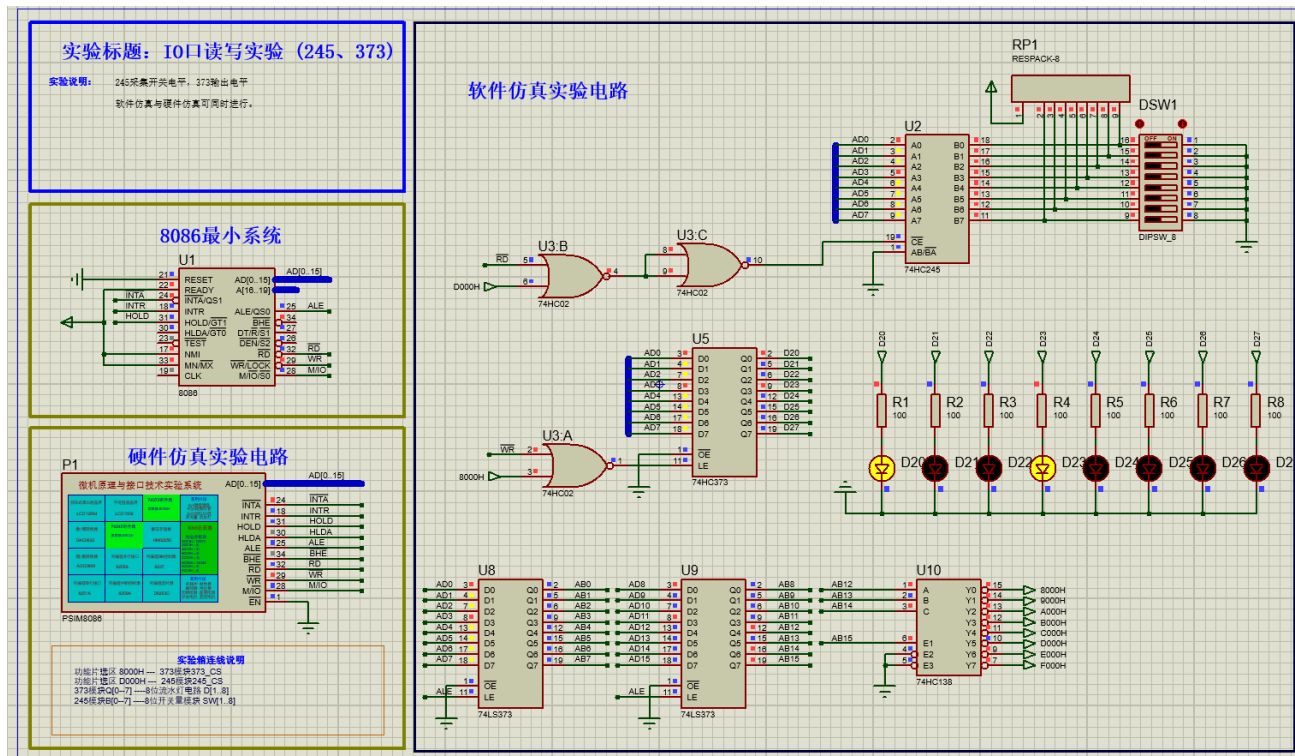


Figure 1: 流水灯电路原理图

输入电路：

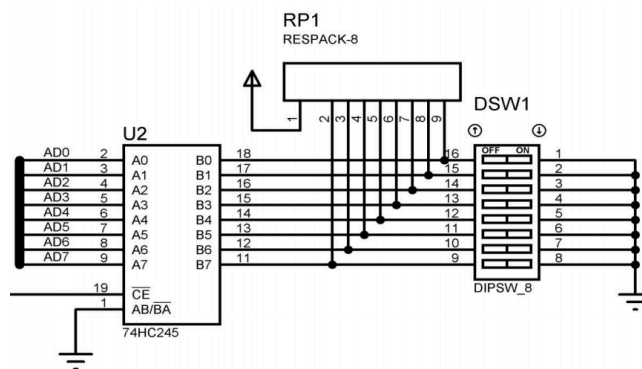


Figure 2: 流水灯输入电路

实验选用 74HC245 作为输入缓冲器，作用是将 8 路开关的机械信号隔离并缓冲后送入数据总线。开关一侧通过上拉电阻（RESPACK-8）接高电平，另一侧接 74HC245 的 B 端口；当 CPU 从地址 D000H 读取数据时，片选信号使能 74HC245，开关状态经缓冲后送至数据总线。

输出电路：

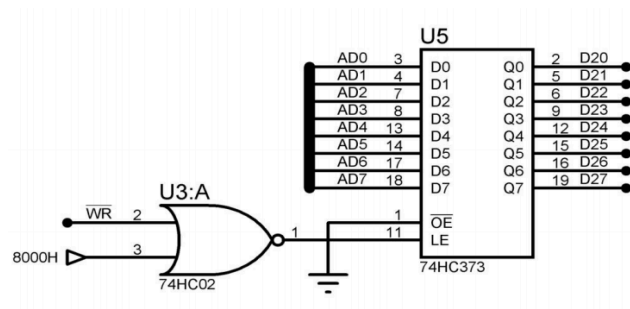


Figure 3: 流水灯输出电路

实验选用 74HC373 作为输出锁存器，作用是将 8086 数据总线上短暂的数据信号锁存并稳定输出至 LED。当 CPU 向地址 8000H 写入数据时，地址译码电路（由 74HC02 或非门实现）产生一个正脉冲，使 74HC373 的锁存使能端（LE）有效，此时数据总线上的 8 位数据被锁存至输出端，驱动 LED 显示。该设计确保了在 CPU 执行其他任务时，LED 状态仍能保持稳定。

```

1      ; 功能：从输入端口（开关）读取数据，立即输出到输出端口（8 个 LED 灯）
2
3      CODE SEGMENT
4          ASSUME CS:CODE
5
6      ; 端口地址定义
7      IN245 EQU 0D000H    ; 输入端口地址
8      OUT373 EQU 8000H    ; 输出端口地址
9
10     START:
11         MOV DX, IN245    ; 设置输入端口地址
12         IN AL, DX        ; 从输入端口读取数据到AL寄存器
13
14         MOV DX, OUT373   ; 设置输出端口地址
15         OUT DX, AL       ; 将AL寄存器数据输出到端口
16
17         JMP START        ; 循环执行
18
19     CODE ENDS
20     END START            ; 程序入口点为START

```

1.2 LCD 字符显示器控制

1.2.1 电路原理图与代码逐行讲解

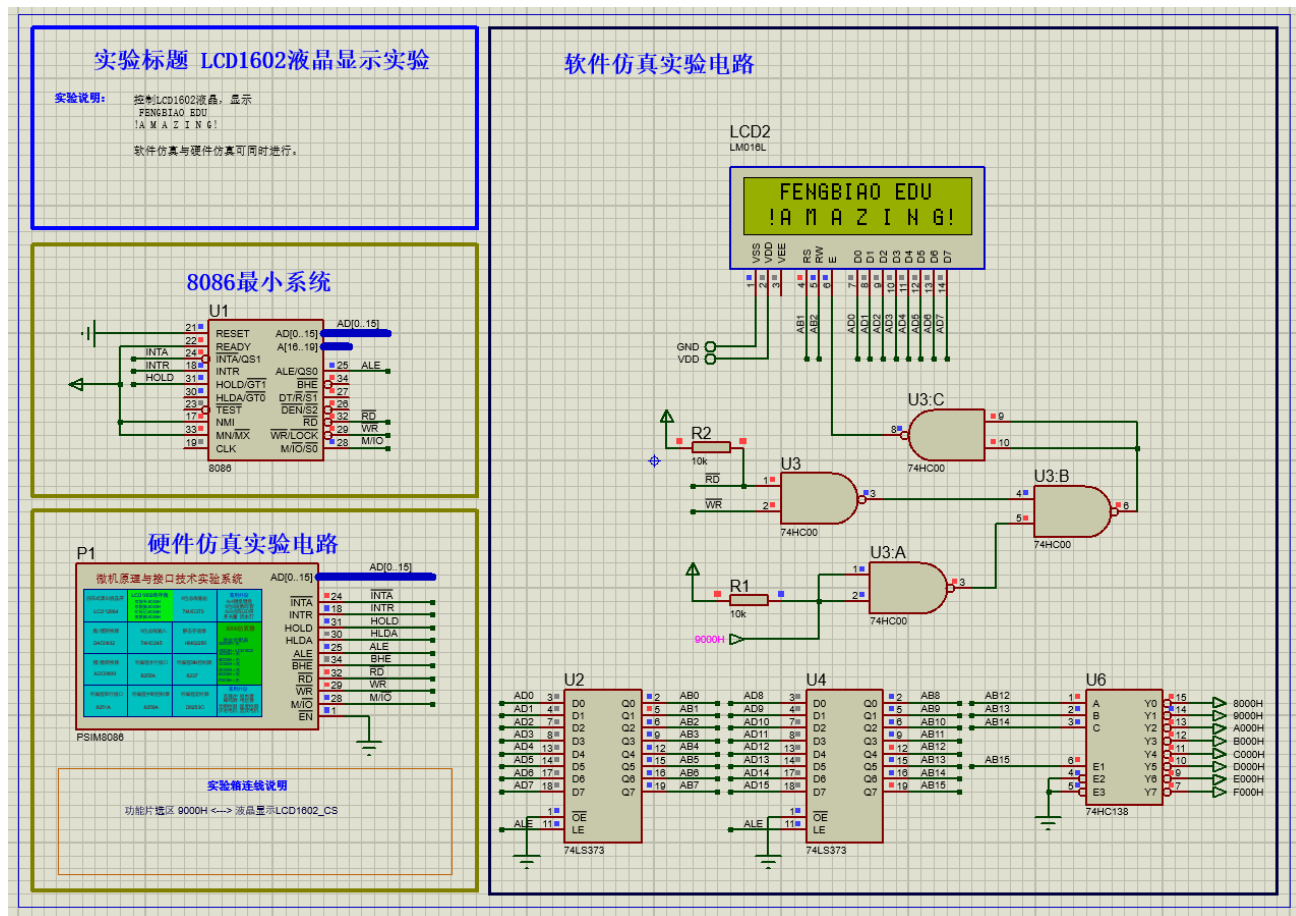


Figure 4: LCD 显示屏电路原理图

```
1 ; 功能：8086控制LCD显示器显示字符串
2 CODE SEGMENT 'CODE' ; 代码段定义
3 ASSUME CS:CODE, DS:DATA, SS:STACK ; 段寄存器关联
4
5 ; 地址由74HC138译码器产生
6 LCD_CMD_WR EQU 9000H ; LCD命令写入端口（对应U5-74HC373）
7 LCD_DATA_WR EQU 9002H ; LCD数据写入端口
8 LCD_BUSY_RD EQU 9004H ; LCD忙状态读取端口
9 LCD_DATA_RD EQU 9006H ; LCD数据读取端口
10
11 START:
12     MOV AX, DATA ; 加载数据段地址
13     MOV DS, AX ; 设置DS指向数据段
14     MOV AX, STACK ; 加载堆栈段地址
15     MOV SS, AX ; 设置SS指向堆栈段
16     MOV AX, TOP ; 获取堆栈顶部地址
17     MOV SP, AX ; 设置堆栈指针SP
18
19     ; IN AX, DX
20
21
22 ; 参考手册初始化流程
23 MOV AX, 30H ; 这条指令没找到，可能是笔误
24 CALL WRCMD ; 写入命令
```

```

25
26     MOV AX, 38H          ; 8位接口, 2行显示
27     CALL WRCMD
28
29     MOV AX, 0CH          ; 开显示, 关光标, 不闪烁
30     CALL WRCMD
31
32     MOV AX, 01H          ; 清屏
33     CALL WRCMD
34
35     MOV AX, 06H          ; 地址自动递增, 显示不移动
36     CALL WRCMD
37
38 MAINLOOP:
39     ; 第一行字符串
40     MOV AX, 80H          ; 设置DDRAM地址: 第一行首地址(80H)
41     MOV CX, 16           ; 16个字符
42     LEA DI, str1
43     CALL WRSTR
44
45     ; 第二行字符串
46     MOV AX, 0C0H        ; 设置DDRAM地址: 第二行首地址(C0H)
47     MOV CX, 16           ; 16个字符
48     LEA DI, str2
49     CALL WRSTR
50
51     ; 清屏并重新开始
52     MOV AX, 01H          ; 清屏
53     CALL WRCMD
54     JMP MAINLOOP        ; 无限循环
55
56 ; 功能: 将命令写入LCD命令寄存器
57 WRCMD PROC
58     MOV DX, LCD_CMD_WR   ; DX指向命令端口
59     OUT DX, AX           ; 输出命令到LCD
60     RET                  ; 返回调用者
61 WRCMD ENDP
62
63 ; 功能: 在当前光标位置写入字符串
64 ; 输入参数: AX = DDRAM地址(行地址)
65 ;           CX = 字符数
66 ;           DI = 字符串首地址
67 WRSTR PROC
68     CALL WRCMD           ; 1. 设置显示位置(行地址)
69     MOV DX, LCD_DATA_WR  ; 2. DX指向LCD数据端口
70
71     WRBIT: MOV AL, [DI]   ; 3. 从内存读取一个字符
72             OUT DX, AL    ; 4. 输出字符到LCD
73             INC DI        ; 5. 指向下一个字符
74             LOOP WRBIT    ; 6. 循环直到CX=0
75
76             RET          ; 返回调用者
77 WRSTR ENDP
78
79 CODE ENDS
80
81 STACK SEGMENT 'STACK'
82     STA DB 100 DUP(?)    ; 定义100字节堆栈空间
83     TOP EQU LENGTH STA   ; 堆栈顶部地址
84 STACK ENDS
85
86 DATA SEGMENT 'DATA'

```

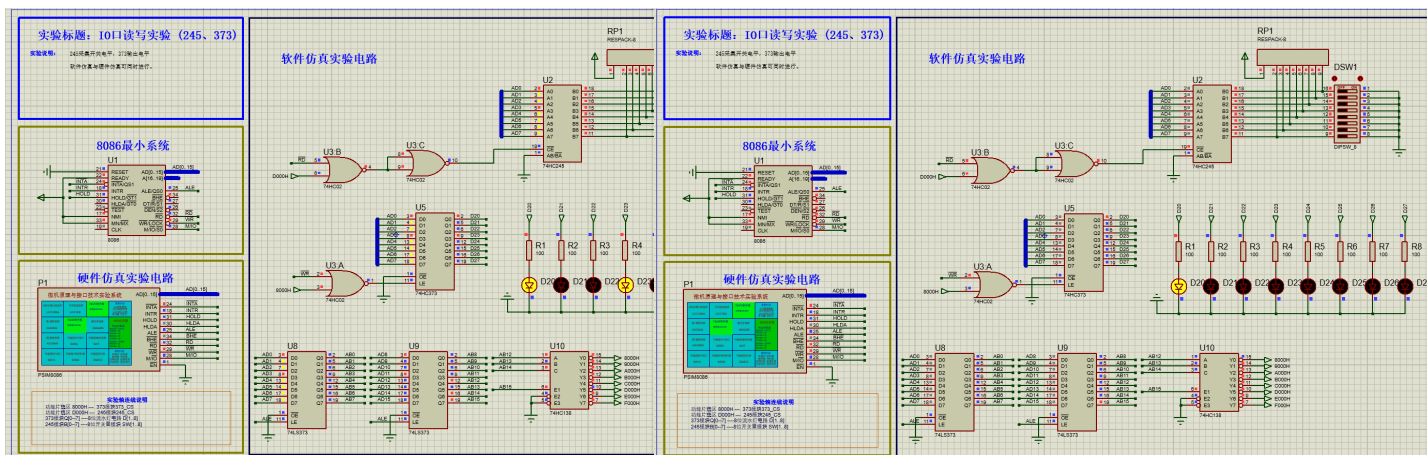
```

87      str1    DB ' FENGBIAO EDU ' ; 第一行显示内容
88      str2    DB ' !A M A Z I N G ! ' ; 第二行显示内容
89      DATA   ENDS
90
91      END     START

```

2 实验结果

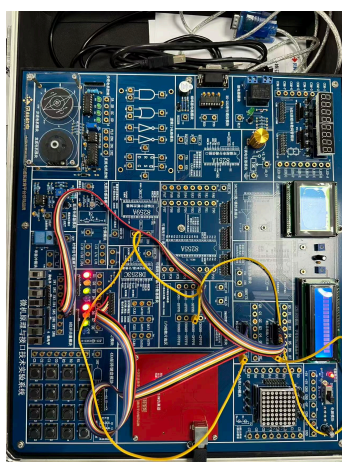
2.1 I/O 端口控制与流水灯



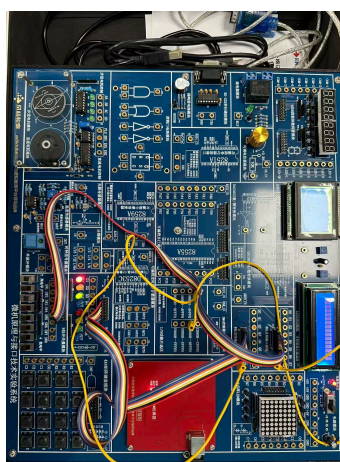
(a) 流水灯软件结果 01

(b) 流水灯软件结果 02

Figure 5: 流水灯软件结果



(a) 流水灯实物结果 01



(b) 流水灯实物结果 02

Figure 6: 流水灯实物结果

2.2 LCD 字符显示器控制

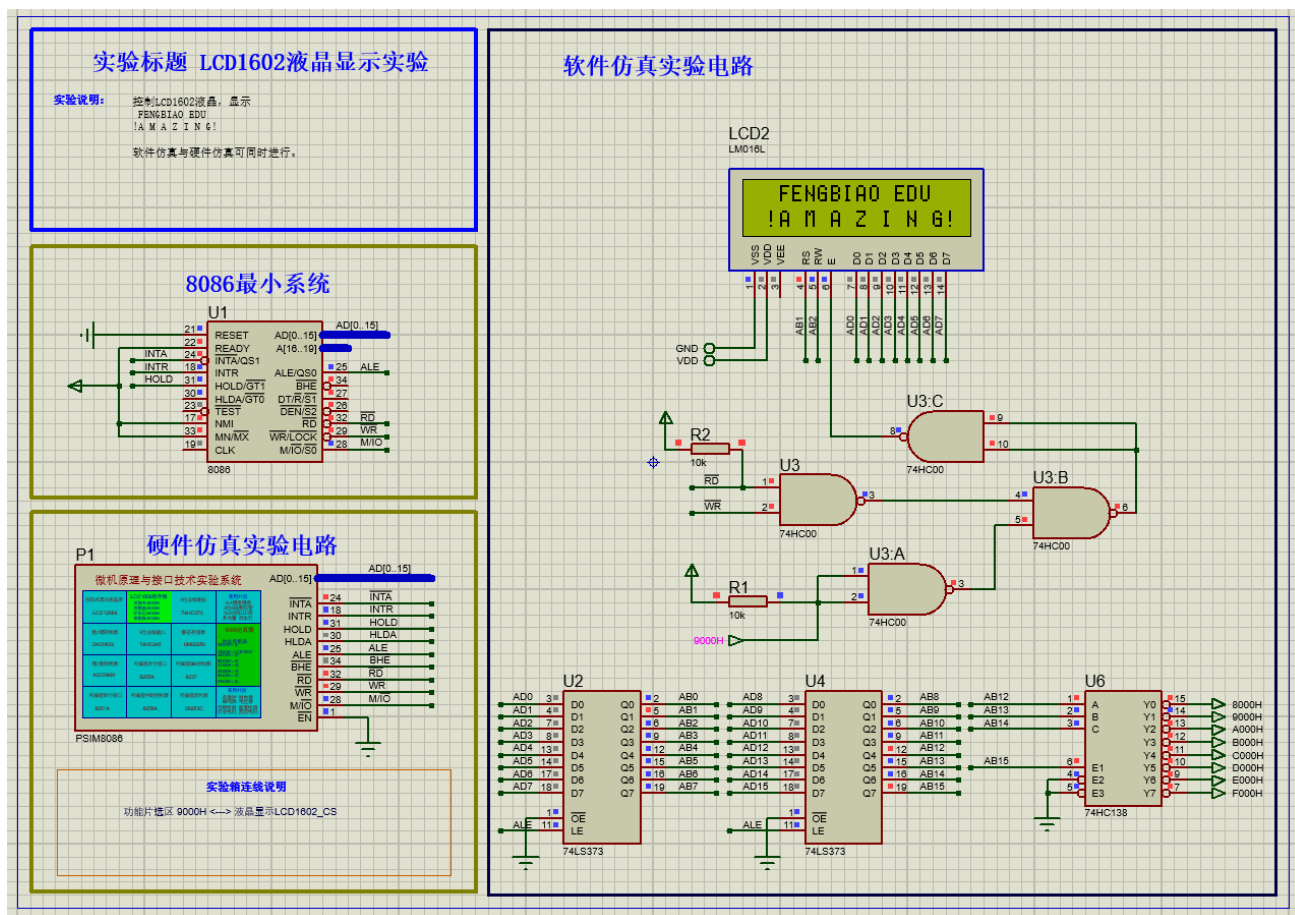


Figure 7: LCD 软件结果

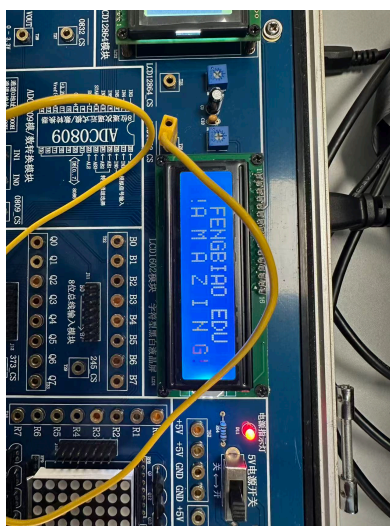


Figure 8: LCD 实物结果

3 思考与总结

3.1 扩展与鲁棒性提升

3.1.1 硬件扩展方案

- (1) **输入防抖电路**: 在拨码开关与 74HC245 之间增加 RC 滤波电路, 硬件消除机械抖动
- (2) **输出驱动增强**: 在 74HC373 输出端增加晶体管或 ULN2003 驱动芯片, 提高 LED 驱动能力
- (3) **多路复用扩展**: 通过 74HC138 扩展更多 I/O 端口, 实现更多外设控制

3.1.2 软件鲁棒性改进

通过软件算法提高系统的稳定性和可靠性, 具体可行代码见附录 A。

3.1.3 故障诊断与保护机制

- (1) **输入校验**: 增加输入范围检查, 过滤非法状态
- (2) **看门狗定时器**: 添加硬件或软件看门狗, 防止程序跑飞
- (3) **输出保护**: 限制输出变化频率, 防止 LED 过流
- (4) **状态反馈**: 通过输入端口回读输出状态, 实现闭环控制

3.2 思考与展望

通过本次实验, 我们不仅掌握了 8086 微处理器的 I/O 编程技术, 更理解了硬件接口设计的基本原则。在未来的系统设计中, 应综合考虑软硬件协同、可靠性设计、扩展性规划等多方面因素, 设计出更加稳定、高效、实用的嵌入式系统。

附录 A 增强型 I/O 控制程序

```

1
2  IN_PORT      EQU 0D000H
3  OUT_PORT     EQU 8000H
4  DEBOUNCE_CNT EQU 5
5
6  DATA        SEGMENT
7      switch_state DB 0
8      led_pattern  DB 0
9      debounce_buf DB 5 DUP(0)
10 DATA        ENDS
11
12 CODE         SEGMENT
13     ASSUME CS:CODE, DS:DATA
14
15 START:
16     MOV AX, DATA
17     MOV DS, AX
18     MOV switch_state, 0
19     MOV led_pattern, 01H
20
21 MAIN_LOOP:
22     CALL READ_SWITCH_DEBOUNCED
23     MOV AL, switch_state
24     AND AL, 0FH
25     CALL SET_LED_MODE
26     MOV AL, led_pattern
27     MOV DX, OUT_PORT
28     OUT DX, AL
29     MOV AL, switch_state
30     SHR AL, 4
31     CALL ADJUST_DELAY
32     JMP MAIN_LOOP
33
34 READ_SWITCH_DEBOUNCED PROC
35     PUSH CX
36     PUSH SI
37     PUSH AX
38     MOV CX, DEBOUNCE_CNT
39     MOV SI, 0
40 SAMPLE_LOOP:
41     MOV DX, IN_PORT
42     IN AL, DX
43     MOV debounce_buf[SI], AL
44     INC SI
45     CALL SHORT_DELAY
46     LOOP SAMPLE_LOOP
47     CALL MAJORITY_VOTE
48     POP AX
49     POP SI
50     POP CX
51     RET
52 READ_SWITCH_DEBOUNCED ENDP
53
54 MAJORITY_VOTE PROC
55     PUSH BX
56     PUSH CX
57     PUSH DX
58     MOV CX, 8
59     MOV BX, 0

```

```

60 BIT_LOOP:
61     MOV DL, 0
62     MOV SI, 0
63 COUNT_LOOP:
64     MOV AL, debounce_buf[SI]
65     SHR AL, CL
66     ADC DL, 0
67     INC SI
68     CMP SI, DEBOUNCE_CNT
69     JL COUNT_LOOP
70     CMP DL, DEBOUNCE_CNT/2 + 1
71     JB BIT_ZERO
72     MOV AL, 1
73     SHL AL, CL
74     OR switch_state, AL
75     JMP NEXT_BIT
76 BIT_ZERO:
77     MOV AL, 0FEH
78     ROL AL, CL
79     AND switch_state, AL
80 NEXT_BIT:
81     INC BL
82     CMP BL, 8
83     JL BIT_LOOP
84     POP DX
85     POP CX
86     POP BX
87     RET
88 MAJORITY_VOTE ENDP
89
90 SET_LED_MODE PROC
91     CMP AL, 0
92     JNE MODE1
93     MOV led_pattern, 0FFH
94     RET
95 MODE1:
96     CMP AL, 1
97     JNE MODE2
98     MOV led_pattern, 00H
99     RET
100 MODE2:
101     CMP AL, 2
102     JNE MODE3
103     MOV AL, led_pattern
104     ROL AL, 1
105     MOV led_pattern, AL
106     RET
107 MODE3:
108     MOV led_pattern, AL
109     RET
110 SET_LED_MODE ENDP
111
112 ADJUST_DELAY PROC
113     PUSH CX
114     PUSH BX
115     INC AL
116     MOV BL, AL
117     MOV BH, 0
118 DELAY_OUTER:
119     MOV CX, 0FFFFH
120 DELAY_INNER:
121     LOOP DELAY_INNER

```

```
122     DEC BX
123     JNZ DELAY_OUTER
124     POP BX
125     POP CX
126     RET
127 ADJUST_DELAY ENDP
128
129 SHORT_DELAY PROC
130     PUSH CX
131     MOV CX, 0FFFFH
132 DELAY_LOOP:
133     LOOP DELAY_LOOP
134     POP CX
135     RET
136 SHORT_DELAY ENDP
137
138 CODE     ENDS
139     END START
```

Listing 1: 增强型 I/O 控制程序